Lab: Features & Matching

Computer Vision Course

Session Roadmap



Part 1: Harris Corner Detection

- Interest Point detection logic
- Parameter exploration (qualityLevel, minDistance)



Part 2: Feature Matching & Lowe's Ratio Test

- Nearest Neighbor matching
- Filtering ambiguous matches using distance ratios



Part 3: Image Stitching (RANSAC)

- Homography estimation
- Robust outlier rejection
- Perspective warping for panoramic composition

Overview of Tasks



Task 1: Harris Corner Parameter Exploration

Goal: Detect 'Interest Points' (corners) invariant to rotation and illumination.

Activity: Implement `cv2.goodFeaturesToTrack` and observe how changing sensitivity thresholds affects corner density.



Task 2: Robust Feature Matching

Goal: Establish correspondence between two images of the same scene.

Activity: Implement Brute-Force KNN matching and filter results using Lowe's Ratio Test to remove false positives.



Task 3: End-to-End Image Alignment

Goal: Combine two overlapping images into a single panorama.

Activity: Compute the Homography matrix using RANSAC and warp the source image.

Submission Requirements & Observation Guide

Submission Requirements

- Code Files: `harris_corner.py`, `ratio_test.py`, `alignment.py`, `main.py`
- Report: `Lab_Report.pdf` (Includes visualizations and analysis questions)

OBSERVATION GUIDE (What you should see)

- **In Task 1 (Harris):** Increasing `qualityLevel` drastically reduces detected corners, leaving only sharpest features. Low `minDistance` causes clusters on the same edge.
- **In Task 2 (Matching):** Without `Ratio Test`, expect a 'spiderweb' of messy lines. After `Ratio Test`, lines should be parallel and consistent.
- **In Task 3 (Stitching):** Successful stitch is seamless. Failure looks like 'ghosting' (duplicate edges) or distorted perspective.

Setup & Execution

Tech Stack



Python 3.8+

Libraries

numpy
(Matrix operations)

opencv-python
(Image processing)

matplotlib
(Visualization)

Execution Command

```
python main.py
```

Project Structure

```
/Lab_Features
  /assets
    image1.png
    image2_a.png
    image2_b.png
    image3_a.png
    image3_b.png
  /output
  alignment.py
  harris_corner.py
  main.py
  ratio_test.py
  README.md
```

Part 1: Harris Corner Parameter Exploration

Code Spotlight

```
cv2.goodFeaturesToTrack(image, maxCorners, qualityLevel, minDistance)
```

Your Task

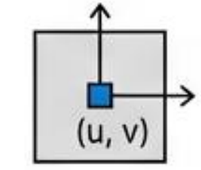
1. Load **assets/checkerboard.jpg** and **assets/scenery.jpg**.
2. Set **maxCorners = 0**. (This forces the algorithm to return all corners meeting the quality threshold).
3. Experiment with **qualityLevel** (Try 0.01 vs 0.1).
4. Visualize the corners using **cv2.circle**.

Hint: Input images must be converted to Grayscale (cv2.cvtColor) before passing them to the Harris detector.

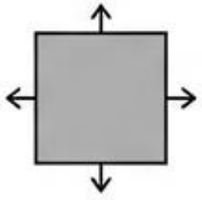
Part 1: Algorithm Flow & Theory

How Harris Works: The Structure Tensor

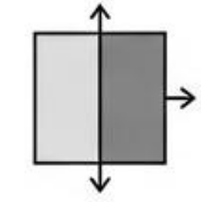
The Logic:



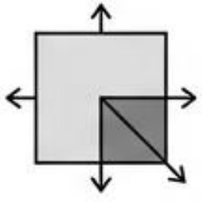
Inspect a small window around a pixel (u, v). Shift the window. Does intensity change?



Flat Region: No change in any direction.



Edge: Change in only one direction.



Corner: Significant change in all directions.

Eigenvalues Determine the Feature

Math Logic:

If $\lambda_1 \approx 0$ and $\lambda_2 \approx 0$ → Flat

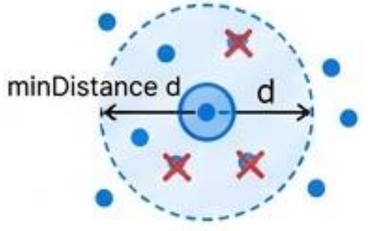
If $\lambda_1 \gg \lambda_2$ → Edge

If λ_1 and λ_2 are both large → Corner

$$\begin{bmatrix} \lambda_1 & \lambda_1 \\ \lambda_2 & \lambda_2 \end{bmatrix}$$

Non-Maximum Suppression (NMS)

The minDistance parameter suppresses neighbors within Euclidean distance 'd', keeping only the local maximum.



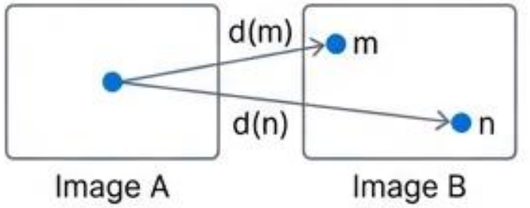
Part 2: Lowe's Ratio Test

Your Task

1. **Compute Descriptors** (e.g., SIFT/ORB) for your Harris corners. (Helper function provided).

```
cv2.SIFT_create()
cv2.ORB_create()
```

2. Perform **KNN Matching** (k=2). For every feature in Image A, find the **top 2 best matches in Image B** (match 'm' and match 'n').



Implement the Ratio Test

$$\text{distance}(m) < \text{threshold} \times \text{distance}(n)$$

Recommended Threshold: **0.75**

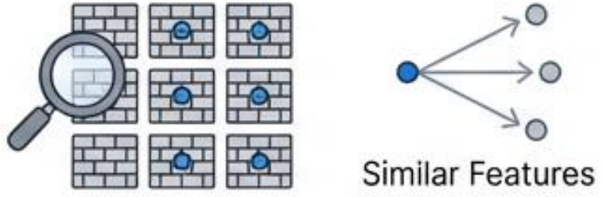
Keep match 'm' **IF AND ONLY IF**: the condition above is met.

Part 2: Theory & Analysis

Why Brute Force Fails

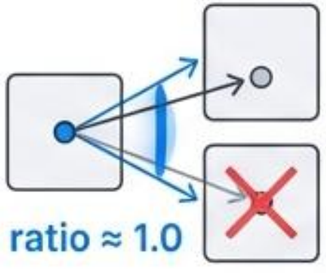
The Ambiguity Problem

In images with repetitive textures (e.g., brick walls, foliage), a feature point looks mathematically similar to many others.



The Solution

If the best match is significantly better than the second-best match, it is likely correct. If they are close (ratio ≈ 1.0), it is ambiguous.



Report Analysis Question

Run your code with thresholds `0.60`, `0.75`, and `0.90`.



- **Question 1:** How does the number of preserved matches change?
- **Question 2:** At 0.90, do you see more false positives (incorrect connections)? Why?

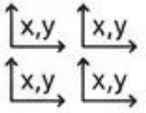
Part 3: End-to-End Image Alignment

Your Task

1 

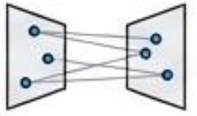
Detect & Match.

Get the filtered "Good Matches" from Task 2.

2 

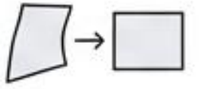
Extract Coordinates.

Separate the source points (`src_pts`) and destination points (`dst_pts`).

3 

Compute Homography.

Use `cv2.findHomography`. Set method to `cv2.RANSAC`. Set `RansacReprojThreshold` to `5.0`.

4 

Warp.

Use `cv2.warpPerspective` to transform the source image into the destination coordinate space.

The Homography matrix H is a 3x3 transformation matrix that maps points from one planar surface to another.

Part 3: Algorithm Flow (RANSAC)

RANSAC: RANdom SAmple Consensus

1 **Sample:** Randomly select 4 pairs of matching points.

2 **Model:** Compute Homography matrix **H** for this subset.

3 **Score:** Project all other points using **H**. Count "**Inliers**" (points threshold).

4 **Loop:** Repeat Repeat for N iterations.



5 **Select:**
Keep the **H** that produced the highest number of Inliers.



Why? Linear regression is sensitive to outliers. RANSAC robustly ignores them.